# PROJECT AND TEAM INFORMATION

## Project Title

**Emergency Call Detection and Priority Routing**

| | |
|---|---|
| *Team Name:*<br>*Team #* | *NOVEXA* |
| **Team member 1 (Team Lead)**<br>*(Last Name, name: student ID: email, picture):* | *Dixit, Avika :2510018349*<br>*avika51110@gmail.com*<br>  |
| **Team member 2**<br>*(Last Name, name: student ID: email, picture):* | *Bisht, Harshita*<br>*:2510360038*<br>*bishtharshi25@gmail.com*<br>  |
| **Team member 3**<br>*(Last Name, name: student ID: email, picture):* | *Mittal, Saksham :*<br>*2510014103*<br>*sakshammittal.geu@gmail.c*<br>  |
| **Team member 4**<br>*(Last Name, name: student ID: email, picture):* | *Bhandari, Nitin Singh*<br>*:2512510026*<br>*nitinsinghbhandari@gmail.c*<br>  |

*m*

# PHASE-II PROJECT PROGRESS DESCRIPTION

## Project Abstract

Our project is about creating an Emergency Call Detection and Priority Routing System.. The main goal of the project is to make the way emergency reports are handled organized and help emergency teams act more quickly. In an emergency, many people might call and report the same situation .Some of these reports might have the details; some might be missing important information and some might even have different details. If each call is treated as a case it can cause confusion and make the job harder for the people handling the calls.

Our system processes incoming emergency reports and checks whether they are related to an incident that has already been reported. If several reports are about the event they can be grouped together instead of being treated as separate cases. The system can also give attention to reports that don't have much useful information. Another important part of the project is assigning priorities to incidents based on their urgency and severity. Incidents are ranked based on how serious they're. Dangerous situations are placed higher in the list so that emergency teams can deal with them first.

The system uses different data structures to manage emergency reports. A queue processes incoming reports in the order they are received.A hash map is used to store and find emergency reports. Graphs and DSU can be used to find links between reports and put them into groups. A priority queue is used to organise incidents according to their urgency. The project is developed in C++ and uses object-oriented programming concepts such as classes, inheritance, and polymorphism, along with the Standard Template Library (STL).

In terms the project takes many emergency reports groups the ones that are related into incidents gives higher priority t to more serious situations and helps the operator decide which emergency to handle first.

## Updated Project Approach and Architecture

The project "Emergency Call Detection & Priority Routing" is currently developed as console based .The main objective of the project focuses on managing incoming emergency reports by identifying duplicate or related calls, combining them into a single incident, and assigning priorities based on urgency. This helps responders focus on critical situations and manage emergencies more efficiently.

The current architecture follows object-oriented approach. Different classes are used for major entities such as EmergencyCall, Incident, Caller, Dispatcher, ResponseTeam, and CallAnalyzer. Data structures are integrated into the system according to their requirements. A Hash Map is used to identify duplicate calls or reports, Graph/Disjoint Set concepts are used to group related calls into incidents, a Priority Queue is used to prioritize critical incidents, and a Queue represents incoming emergency calls.

The system till now has a fixed given inputs for testing . The CallAnalyzer is the part that handles communication with other classes. It analyzes calls. Updates incidents accordingly. The Dispatcher takes the processed data. Determines the priority level of each incident based on its urgency and severity.Then it assigns the response teams based on the urgency.

As the current version is a local console-based prototype, no external communication protocol or network , API is required at this stage.

In Phase 3, the architecture will be extended to accept dynamic user input and provide a more interactive system

## Tasks Completed

| Task Completed | Team Member |
|---|---|
| Project Design and System Workflow | Saksham Mittal |
| Data Structure and Incident Grouping Implementation | Avika Dixit |
| Emergency Report Handling and Call Analysis Implementation | Harshita Bisht |
| Incident Prioritization and Response Team Implementation | Nitin Singh Bhandari |

## Challenges/Roadblocks

During Phase 2 many difficulties appeared while the project was being developed. The biggest problem was to combine data structures for example Hash Map, Queue, Priority Queue and Graph/Disjoint Set into one system. Also, the coordination of communication between various C++ classes, including EmergencyCall, Incident, CallAnalyzer, Dispatcher, and ResponseTeam, required a lot of attention.

Furthermore, there were challenges with compilation and class-specific errors during development. Moreover, the implementation of the logic for identifying emergency calls and dispatching related reports into incidents required a lot of attention and effort to ensure the accuracy of the identification method and relationship logic

.

Adding priority-based handling of emergency incidents needed an understanding of how to use a Priority Queue in the project. Because Phase 2 aims to show the core functions of the project the system now uses inputs. Changing the system to take dynamic user input and to be more interactive is planned for Phase 3.

## Tasks Pending

| Task Pending | Team Member |
|---|---|
| Making the system dynamic instead of static. | Saksham Mittal |
| Connecting the system features so that reports and incidents can be updated. | Nitin Singh Bhandari |
| Working on the UI/UX and designing the dashboard. | Avika Dixit |
| Improving the dashboard layout, colours, and overall look. | Harshita Bisht |

# Project Outcome/Deliverables

The main outcome of our project is an Emergency Call Detection and Priority Routing System that helps emergency services manage incoming reports more efficiently.

The key deliverables are:

1.Emergency Report Management: A FIFO queue-based system to receive and process emergency reports in a manner.

2.Low-Information Call Detection: A call analyzer to identify incomplete or low information reports.

3.Incident Grouping: Use of hash maps, graphs and DSU to identify and combine related reports into a single incident.

4.Incident Prioritization: A priority queue decides which emergency ncident should be handle first.

5.Response Management: A system workflow that helps determine which incidents should be handled first and support team dispatch the teams.

6.C++ Implementation: Integration of the system ,into the existing infrastructure.

# Progress Overview

The project has made progress during Phase 2. The basic system design has been done . The main classes have been written in C++. The required data structures, including Hash Map, Queue and Priority Queue have also been included for handling and processing emergency reports.

The current prototype was tested with a pre-established set of procedures, which enables the selection of input data, fixed during the trial. The model was produced using the test data to observe the system's working capabilities. A basic version of the system was developed to demonstrate its main features.
. It allows us to focus on the flow without dealing with real-time data. It can show emergency reports find related reports put them into incidents give them priorities and help with the basic dispatch process.

The main OOP structure and the way the major components talk to each other have been finished. The project is still on schedule with the Phase 2 requirements, with the static-input prototype being used to show and test the features.

In Phase 3, we plan to add dynamic inputs, improve user interaction, and test the system more thoroughly. These features will be added after the main logic and data-structure implementation have been fully tested.

# Codebase Information

**Programming Language:** C++

**IDE:** Visual Studio Code

**Version Control:** Git and GitHub

**Repository:** https://github.com/avikad05/PBL-3RD-SEM

**COMMITS**

Caller.h – Stores caller details ,including caller ID,name and phone number.

EmergencyCall.h – Contains information about each emergency call for example location, type, de-scription, severity and time.

Incident.h – Represents a consolidated incident with related emergency calls and their priority information.

DSU.h – Uses Disjoint Set Union structure for grouping related emergency calls into the same incident.

CallAnalyzer.h – Checks calls for relation, analyzes calls, identifies low-information reports, and calculates incident priority.

ResponseTeam.h – Declares different emergency response teams, such as Ambulance, Police and Fire teams.

Dispatcher.h - Chooses and assigns the right response team basedon the type of emergency.

Static Main.cpp – Contains the main execution of the project with predefined emer-gency call data. It takes care of the calls through the queue, bundles related calls into incidents, sets their priorities and assigns the right response team.

# Testing and Validation Status

| Test Type | Status (Pass/Fail) | Notes |
|---|---|---|
| **Queue Testing** | **Pass** | Incoming emergency calls are processed in FIFO order. |
| **Duplicate call detection** | **Pass** | Hash map identifies calls with matching emergency types and locations |
| **Incident grouping** | **Pass** | DSU groups related emergency calls based on defines conditions |
| **Low information detection** | **Pass** | Calls with missing insufficient information are flagged |
| **Priority queue testing** | **Pass** | Higher priority incidents are given precedence processing |
| **Response team dispatching** | **Pass** | Appropriate teams are selected according to the emergency type. |

## Deliverables Progress

| DELIVERABLE | STATUS |
|---|---|
| Handling of Emergency Reports using Queue | COMPLETED |
| Analysis of Reports and Identification of Related Reports | COMPLETED |
| Incident Grouping using Hash Map, Graph and DSU | IN PROGRESS |
| Incident Prioritization using Priority Queue | COMPLETED |
| Team and Dispatch Process | IN PROGRESS |
| Working Prototype – Static Input | COMPLETED |
| Dynamic Input Handling | IN PROGRESS |
| Final Testing and Integration | IN PROGRESS |
| UI/UX dashboard | IN PROGRESS |